

MERN STACK PROJECT

CivicDesk

Online Complaint Registration
& Management System

Presented by

Subodh Shekhar

Full Stack Developer · MERN Stack

Subodh Shekhar



ROLE

Full Stack Developer (MERN)



STACK

MongoDB · Express · React · Node.js



FOCUS

Scalable Web Applications



PROJECT

CivicDesk — Complaint Management



Core Skills:

React.js

Node.js

MongoDB

Express

JWT Auth

REST API

What is CivicDesk?

CivicDesk is a full-stack MERN web application that streamlines the entire complaint lifecycle — from submission to resolution — with real-time tracking, role-based access, and intelligent routing for citizens, agents, and administrators.



Citizen Portal

File, track & chat on complaints in real time directly from any device.



Agent Workspace

Manage custom queues, update status, provide resolution notes & communicate.



Admin Control

Assign specific agents, monitor resolution system health & track analytic metrics.

What We Set Out to Achieve



Centralized Platform

One unified system for registering, tracking and resolving complaints — replacing manual, fragmented traditional methods.



Data-Driven Insights

Admin dashboards surface real-time complaint trends, individual performance statistics and resolution KPIs.



Secure Role-Based Access

JWT authentication with distinct strict privileges for Citizens, Agents and Admins ensuring system data privacy.



Responsive Design

Fully responsive optimized React UI layout layer ensuring seamless user experience across mobile and desktop.



Real-Time Transparency

Automated updates and live custom status timelines keep citizens fully informed at every single stage.



Scalable Architecture

MERN infrastructure with MongoDB indexing, optimized query lookups and clean modular controller structures.

Built on the MERN Stack



MongoDB

DATABASE

NoSQL document store • Fast indexed structural queries • Schema validation via Mongoose ODM



Express.js

BACKEND

Robust RESTful API design router • Security & custom logging middleware • Global error filters



React.js

FRONTEND

Component-driven interface architecture • Central state management Context • Axios client layer



Node.js

RUNTIME

Asynchronous highly-performant core environment • JWT auth utilities • NodeMailer mailing logic

Also uses: JWT • bcryptjs • nodemailer • Bootstrap/MUI • Helmet • CORS protection • express-validator • dotenv configuration

Core Features of the System



JWT Authentication

- Secure stateless login tokens for 3 core roles
- Robust password hashing via bcryptjs library
- Granular server route middleware protection



Smart Case Routing

- Automatic routing mapped by category types
- Intelligent agent pairing queues thresholds
- SLA threshold tracking and urgency escalation



Real-Time Tracking

- Interactive 4-stage processing timeline view
- Clean progress bar tracking layout visual
- Immediate status transition alert pings



Agent-Citizen Chat

- Integrated text messaging module channels
- Isolated threads pinned inside each incident
- Permanent audit trail history record logging



Admin Analytics

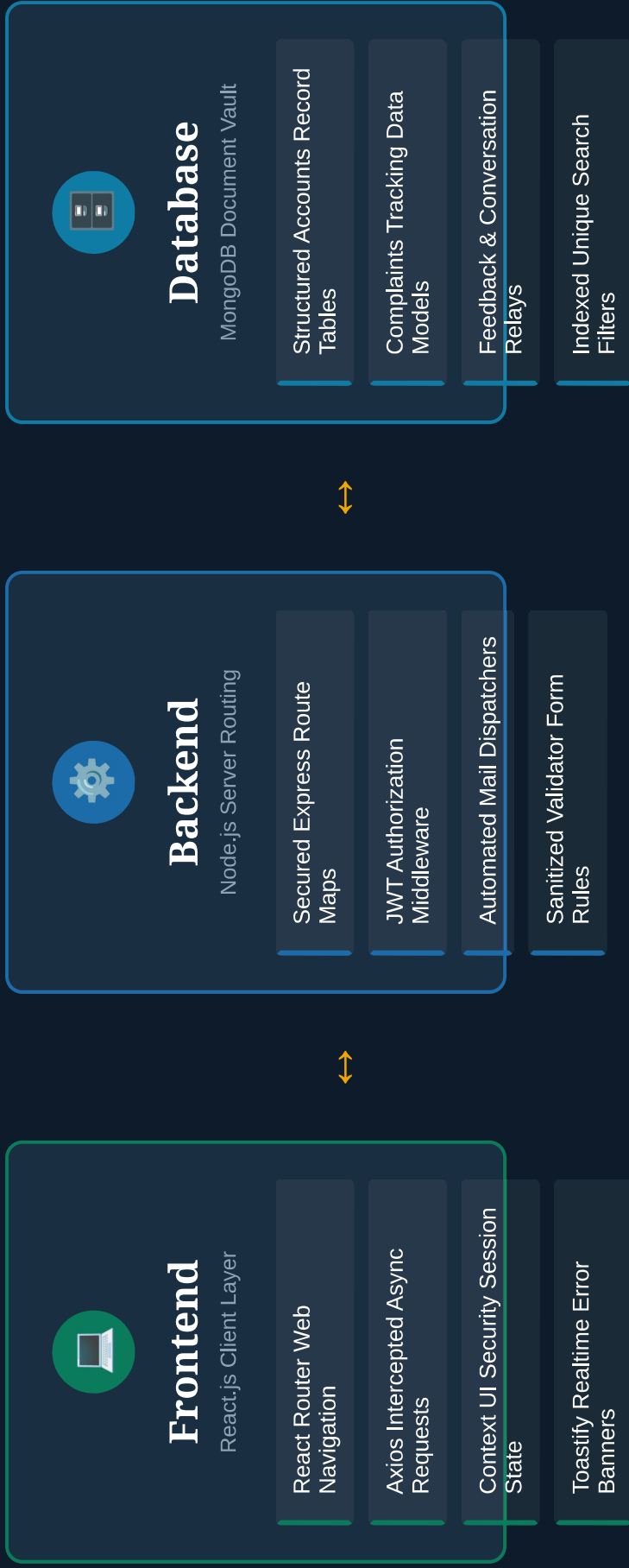
- High-level overall operational health monitors
- Agent throughput rate & volume distributions
- Categorical trends graph breakdowns metrics



Feedback Loop

- Interactive 5-star score evaluation stars
- Citizen narrative resolution review text fields
- Aggregated rolling average satisfaction indicators

3-Tier Full-Stack Architecture



What the Completed System Delivers

3

User Roles

Citizen, Dedicated Staff Agent,
and Platform Administrator

6+

Core Modules

Auth, Logging, Queues, Live Chat,
Aggregates, System Mailers

100%

Full CRUD

Data entries flow seamlessly
without data orphan hazards

JWT

Protected

Endpoints verified and isolated
behind strict controller filters

✓ **Better Velocity:** Minimizes turnaround times using programmatic ticket assignments and clear accountability frameworks.

✓ **Total Clarity:** Increases community trust indices via live multi-stage graphical feedback track trackers.

✓ **Analytics Control:** Empowers executive admin panels to pinpoint bottlenecks with descriptive volume heat tracking maps.

✓ **Hardened Design:** Ensures production readiness via structured environments schema protections and cross-origin security configurations.

What I Overcame & Learned

⚡ Key Project Challenges

- Configuring dynamic granular endpoint permissions mapping to independent session types securely.
- Preventing race structural states during automated high-concurrency ticket distributions.
- Synchronizing structural UI refresh cycles during live reactive background communications cascades.
- Optimizing relational aggregation pipelines effectively across deep collections lookups.
- Sustaining pristine asset uploads paths while preserving tight CORS constraints configurations.

✓ Strategic Takeaways

- Architected high-standard complex distributed systems patterns end-to-end flawlessly.
- Mastered REST controller encapsulation schemas featuring error guards and validation barriers.
- Leveraged top-tier layout patterns and clean state dispatching strategies natively inside React.
- Acquired deep insight on index selection optimization matrices across NoSQL data pools.
- Adopted proper DevOps production configurations workflows adhering to global security compliance standards.